Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 1

Разработка программ с использованием принципов SRP и OCP

Вариант 3 Система Больница

Группа: _____ Студент: _____

Задание на лабораторную работу

Пациенту назначается лечащий Врач. Врач может сделать назначение Пациенту (процедуры, лекарства, операции). Медсестра или другой Врач выполняют назначение. Пациент может быть выписан из Больницы по окончании лечения, при нарушении режима или при иных обстоятельствах.

Необходимо разработать диаграмму классов и диаграмму последовательности, консольное приложение на C# с использованием SRP и OCP и предусмотреть тестирование функциональности объектов.

Цель работы

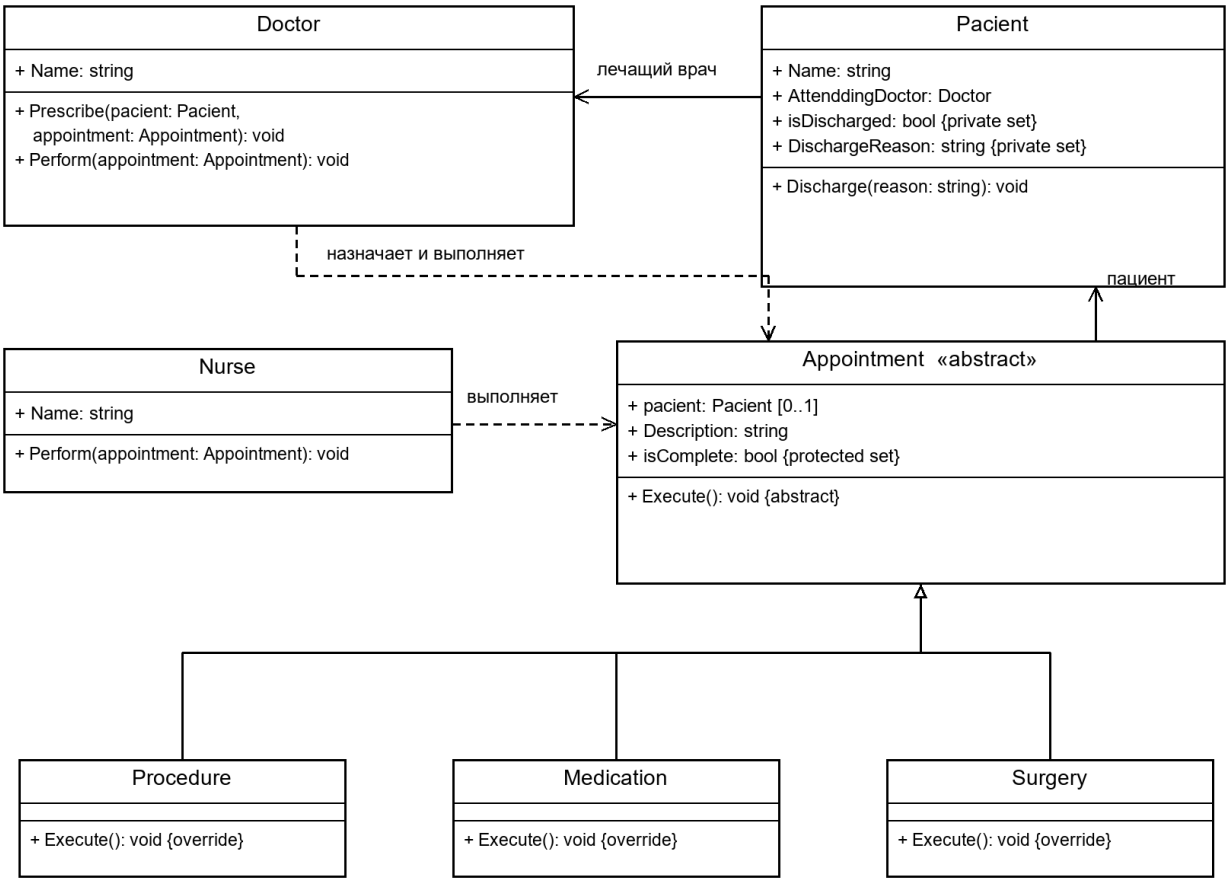
Получить навыки разработки программ с использованием принципа единственной ответственности и принципа открытости и закрытости.

Описание решения

Разработана учебная модель больницы. Лечащий врач связывает назначение с пациентом. Медсестра или другой врач выполняет назначение. При выполнении проверяются наличие пациента и статус назначения. Повторное выполнение запрещено. Метод Discharge сохраняет статус выписки и ее причину.

Программа создана на C# для .NET 10. Все классы расположены в Program.cs. Названия классов и свойств на диаграммах сохранены в соответствии с исходным кодом.

Диаграмма классов



Сплошная стрелка — связь объектов; пунктир — использование.

Пустой треугольник направлен к родительскому классу.

Рисунок 1. Классы предметной области и связи между ними.

Appointment является абстрактным классом. Его наследники реализуют Execute через override. Свойство pacient может отсутствовать до назначения лечения. Program содержит точку входа, консольный вывод и проверки; на диаграмме показаны классы предметной области.

Диаграмма последовательности

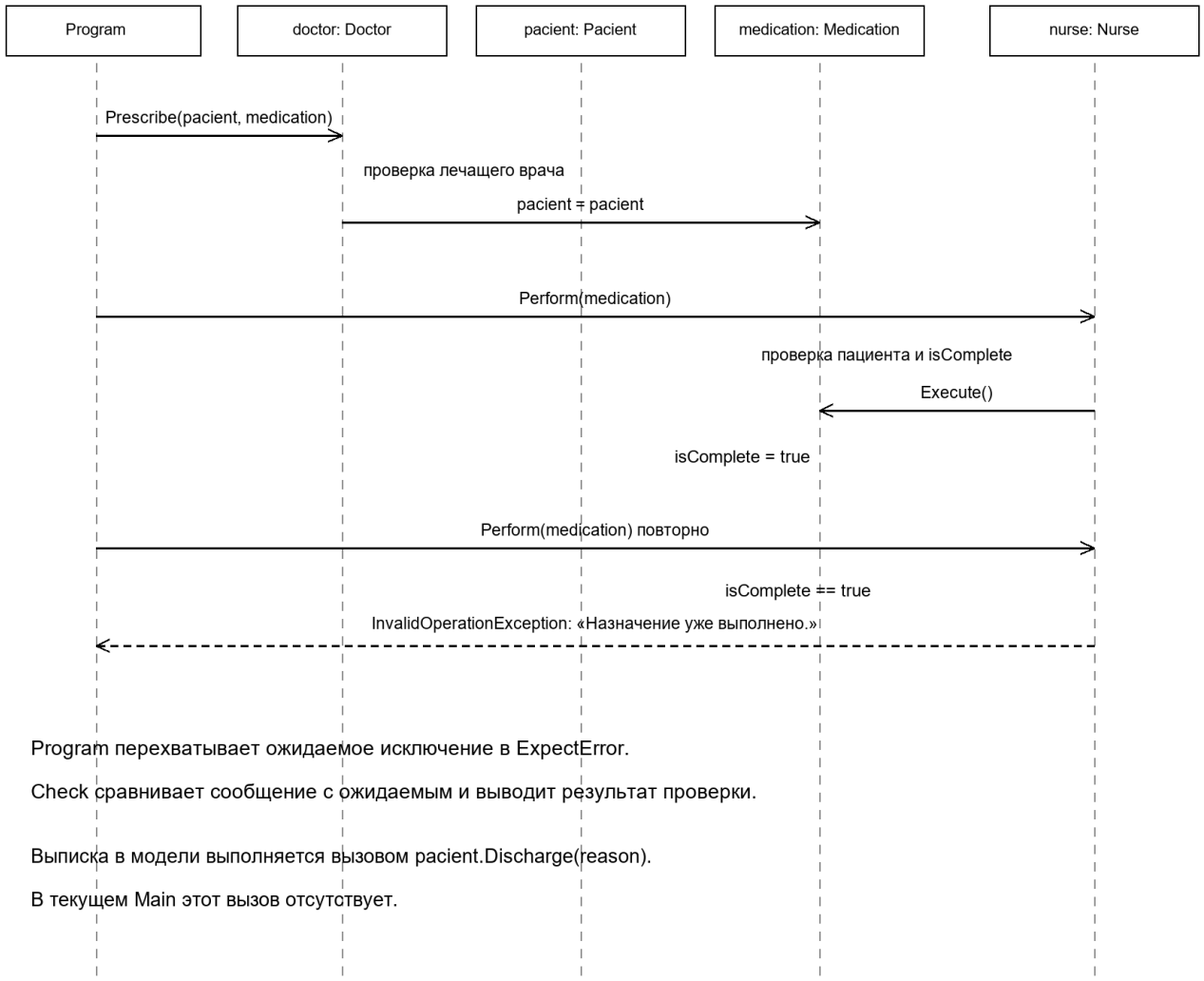


Рисунок 2. Назначение лекарства, выполнение медсестрой и отказ при повторном выполнении.

Линии жизни обозначают объекты. Сплошные стрелки показывают вызовы и запись свойства; пунктирная стрелка показывает передачу исключения в вызывающий код.

Реализация принципа SRP

SRP означает, что класс имеет одну связную ответственность и одну причину для изменения. Количество методов само по себе не определяет соблюдение принципа.

Pacient хранит сведения о пациенте и управляет его состоянием выписки. Doctor реализует действия врача, Nurse выполняет назначения. Appointment задает общие данные и контракт назначения, а Procedure, Medication и Surgery реализуют выполнение конкретных видов лечения.

Консольный вывод отделен от выполнения: Execute изменяет только isComplete. Сообщения и проверки находятся в Program. Поэтому изменение формата сообщений не требует изменения классов назначений. Для небольшой учебной программы демонстрация и проверки объединены в Program.

Реализация принципа OCP

OCP означает возможность расширить поведение программы, сохранив существующий общий код. Doctor.Perform и Nurse.Perform принимают Appointment и вызывают Execute без проверки конкретного типа назначения.

Для добавления обследования можно создать новый класс Examination, унаследовать его от Appointment и реализовать Execute. Методы Perform изменять не потребуется. В Main нужно лишь создать и передать новый объект. Это пример полиморфизма: реализация Execute выбирается по фактическому типу объекта.

Особенности текущей модели

Выполнение всех трех видов назначений моделируется установкой isComplete в true. Медицинские действия не моделируются подробно. Текст причины выписки задается вызывающим кодом. В текущем Main нет демонстрации выписки, хотя метод реализован.

В модели пока нет запрета лечения после выписки и защиты от изменения пациента у существующего назначения. Публичный Execute можно вызвать напрямую в обход Perform. Эти ограничения не проверяются имеющимися тестами.

Тестирование программы

Выполнен запуск dotnet run. В консоли получено 13 успешных проверок, сообщений [ОШИБКА] нет. Проверки реализованы в Program без отдельного тестового фреймворка.

№	Проверка	Результат
1	Назначение связано с нужным пациентом	Пройдена
2	Медсестра выполнила назначение	Пройдена
3	Другой врач выполнил назначение	Пройдена
4	Операция выполнена	Пройдена
5	Чужой врач не может назначить лечение	Пройдена
6	После отказа назначение осталось без пациента	Пройдена
7	Медсестра не может выполнить назначение без пациента	Пройдена
8	Врач не может выполнить назначение без пациента	Пройдена
9	После отказов назначение не выполнено	Пройдена
10	Лечащий врач не может выполнить своё назначение	Пройдена
11	После отказа лечащему врачу назначение не выполнено	Пройдена
12	Медсестра не может повторно выполнить назначение	Пройдена
13	Врач не может повторно выполнить назначение	Пройдена

Check принимает название и логическое условие, выводит результат и увеличивает passed или failed. ExpectError запускает переданный Action внутри try. Если исключения нет, проверка считается неуспешной. При InvalidOperationException сравнивается текст сообщения с ожидаемым.

Дополнительно проверяется неизменность состояния после отказа. Исключение другого типа не перехватывается. Счетчики присутствуют, но текущий Main не выводит их итог и не устанавливает ненулевой код завершения при провале Check.

Исходный код программы

```
namespace Lab1_SRP_OCP
{
    class Pacient
    {
        public required string Name { get; set; }
        public required Doctor AttenddingDoctor { get; set; }

        public bool isDischarged { get; private set; } = false;
        public string DischargeReason { get; private set; } = "";
        public void Discharge(string reason)
        {
            isDischarged = true;
            DischargeReason = reason;
        }

    }
    class Doctor
    {
        public required string Name { get; set; }
        public void Prescribe(Pacient pacient, Appointment appointment)
        {
            if (pacient.AttenddingDoctor != this)
            {
                throw new InvalidOperationException("Назначение может дать только лечащий врач");
            }
            appointment.pacient = pacient;
        }
        public void Perform(Appointment appointment)
        {
            if (appointment.pacient == null)
            {
                throw new InvalidOperationException(
                    "Назначение ещё не связано с пациентом.");
            }

            if (appointment.pacient.AttenddingDoctor == this)
            {
                throw new InvalidOperationException(
                    "Назначение должен выполнить другой врач.");
            }

            if (appointment.isComplete)
            {
                throw new InvalidOperationException(
                    "Назначение уже выполнено.");
            }

            appointment.Execute();
        }
    }
    class Nurse
    {
        public required string Name { get; set; }
        public void Perform(Appointment appointment)
        {
            if (appointment.pacient == null)
```

Исходный код программы продолжение

```
            {
                throw new InvalidOperationException(
                    "Назначение ещё не связано с пациентом.");
            }

            if (appointment.isComplete)
            {
                throw new InvalidOperationException(
                    "Назначение уже выполнено.");
            }

            appointment.Execute();
        }

    }
    abstract class Appointment
    {
        public Pacient? pacient { get; set; }
        public required string Description { get; set; }
        public bool isComplete { get; protected set; } = false;
        public abstract void Execute();

    }
    class Procedure : Appointment
    {
        public override void Execute()
        {
            isComplete = true;
        }
    }
    class Medication : Appointment
    {
        public override void Execute()
        {
            isComplete = true;
        }
    }
    class Surgery : Appointment
    {
        public override void Execute()
        {
            isComplete = true;
        }
    }
    internal class Program
    {
        static int passed;
        static int failed;
        static void Check(string name, bool condition)
        {
            if (condition)
            {
                passed++;
                Console.WriteLine($"[OK] {name}");
            }
            else
```

Исходный код программы продолжение

```
        {
            failed++;
            Console.WriteLine($"[ОШИБКА] {name}");
        }
    }

    static void ExpectError(string name, Action action, string expectedMessage)
    {
        try
        {
            action();
            Check(name, false);
        }
        catch (InvalidOperationException exception)
        {
            Check(name, exception.Message == expectedMessage);
            Console.WriteLine($"  Причина отказа: {exception.Message}");
        }
    }
    static void Main(string[] args)
    {
        var doctor = new Doctor { Name = "Иван Петрович" };
        var otherDoctor = new Doctor { Name = "Алексей Сергеевич" };
        var nurse = new Nurse { Name = "Анна" };

        var pacient = new Pacient
        {
            Name = "Паша",
            AttenddingDoctor = doctor
        };

        var medication = new Medication
        {
            Description = "Приём лекарства"
        };

        doctor.Prescribe(pacient, medication);
        Check("Назначение связано с нужным пациентом",
            medication.pacient == pacient);

        nurse.Perform(medication);
        Console.WriteLine($"Назначение «{medication.Description}» выполнено.");
        Check("Медсестра выполнила назначение", medication.isComplete);

        Console.WriteLine(
            $"Лекарство выдано: {medication.isComplete}");

        var procedure = new Procedure
        {
            Description = "Лечебная процедура"
        };

        doctor.Prescribe(pacient, procedure);

        otherDoctor.Perform(procedure);
        Console.WriteLine($"Назначение «{procedure.Description}» выполнено.");
```

Исходный код программы продолжение

```
            Check("Другой врач выполнил назначение", procedure.isComplete);

            Console.WriteLine(
                $"Процедура выполнена: {procedure.isComplete}");

            var surgery = new Surgery { Description = "Операция" };
            doctor.Prescribe(pacient, surgery);
            otherDoctor.Perform(surgery);
            Check("Операция выполнена", surgery.isComplete);

            var unassigned = new Medication { Description = "Новое лекарство" };
            ExpectError("Чужой врач не может назначить лечение",
                () => otherDoctor.Prescribe(pacient, unassigned),
                "Назначение может дать только лечащий врач");
            Check("После отказа назначение осталось без пациента",
                unassigned.pacient == null);

            ExpectError("Медсестра не может выполнить назначение без пациента",
                () => nurse.Perform(unassigned),
                "Назначение ещё не связано с пациентом.");
            ExpectError("Врач не может выполнить назначение без пациента",
                () => otherDoctor.Perform(unassigned),
                "Назначение ещё не связано с пациентом.");
            Check("После отказов назначение не выполнено", !unassigned.isComplete);

            doctor.Prescribe(pacient, unassigned);
            ExpectError("Лечащий врач не может выполнить своё назначение",
                () => doctor.Perform(unassigned),
                "Назначение должен выполнить другой врач.");
            Check("После отказа лечащему врачу назначение не выполнено",
                !unassigned.isComplete);

            ExpectError("Медсестра не может повторно выполнить назначение",
                () => nurse.Perform(medication),
                "Назначение уже выполнено.");
            ExpectError("Врач не может повторно выполнить назначение",
                () => otherDoctor.Perform(procedure),
                "Назначение уже выполнено.");

        }

    }
}
```

Вывод

В ходе лабораторной работы разработана консольная модель системы Больница. Реализованы назначение лечения лечащим врачом, выполнение назначения медсестрой или другим врачом и метод выписки с сохранением причины. Для трех видов назначений использованы наследование и переопределение абстрактного метода.

Разделение состояния объектов, выполнения назначений и консольного вывода демонстрирует SRP. Работа исполнителей с общим типом Appointment позволяет добавлять новые виды назначений без изменения методов Perform и демонстрирует OCP. Текущие 13 проверок успешно подтверждают рассмотренные успешные и ошибочные сценарии.